

Shiv Nadar University Chennai

Name & Roll Number	Vinay Sathish – 24011101126
Degree & Branch	B.Tech AI & Data Science - Section B (Semester V)
Subject Code & Name	CS3807 – Deep Learning Laboratory (AY: 2026–27)

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

Class distribution in the dataset: 762 Authentic (0) samples and 610 Forged (1) samples.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Result: The dataset was loaded successfully with **1372 rows and 5 columns** (4 features + class label). No missing values were found in any column.

Statistic	Variance	Skewness	Curtosis	Entropy
Mean	0.4337	1.9224	1.3976	-1.1917
Std. Dev.	2.8428	5.8690	4.3100	2.1010
Min	-7.0421	-13.7731	-5.2861	-8.5482
25%	-1.7730	-1.7082	-1.5750	-2.4135
50% (Median)	0.4962	2.3197	0.6166	-0.5867
75%	2.8215	6.8146	3.1793	0.3948
Max	6.8248	12.9516	17.9274	2.4495

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

(Figures and interpretations are provided in Section 7.1.)

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Result: All four features were standardized using **StandardScaler** (zero mean, unit variance). The dataset was split using a stratified 80/20 split:

- Training set size: **1097** samples
- Testing set size: **275** samples

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Result: A Single Layer Perceptron was implemented entirely from scratch (no `sklearn` used for the core model): weights were initialized from $\mathcal{N}(0, 0.01)$, bias was initialized to 0, the step activation function was used, forward propagation computed $z = \mathbf{w}^T \mathbf{x} + b$, and the online perceptron learning rule updated weights sample-by-sample.

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Result: The model was trained with learning rate $\eta = 0.01$ for a maximum of 100 epochs. The epoch-wise learning log is provided in Section 8. The training error did not reach zero within 100 epochs; it decreased sharply during the first 20 epochs and then oscillated between roughly 11 and 19 misclassified samples, indicating the dataset is approximately, but not perfectly, linearly separable.

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Result: On the held-out test set (275 samples):

Metric	Value
Accuracy	0.9891
Precision	0.9837
Recall	0.9918
F1-score	0.9878

Confusion Matrix:

	Predicted Authentic (0)	Predicted Forged (1)
Actual Authentic (0)	151	2
Actual Forged (1)	1	121

Out of 275 test samples, only 3 were misclassified (2 false positives, 1 false negative).

7. Mandatory Plots

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

Students should provide a brief interpretation (2–3 sentences) for every generated plot.

7.1 Generated Plots and Interpretation

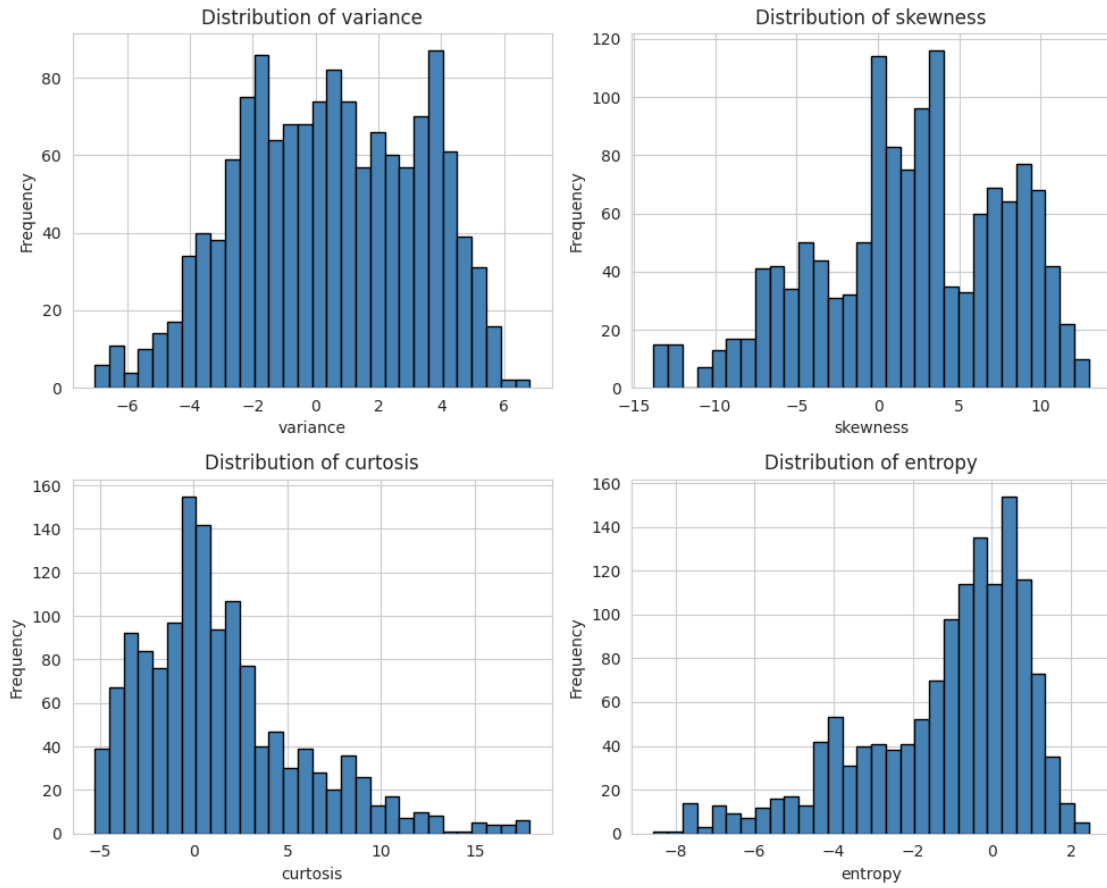


Figure 1: *

Figure 1: Feature histograms

Interpretation: The variance is bell shaped with slight skewness while skewness, kurtosis and entropy show visible asymmetric distributions with stronger tails. Whereas kurtosis has uneven distribution due to presence of higher values on the rightmost tail.

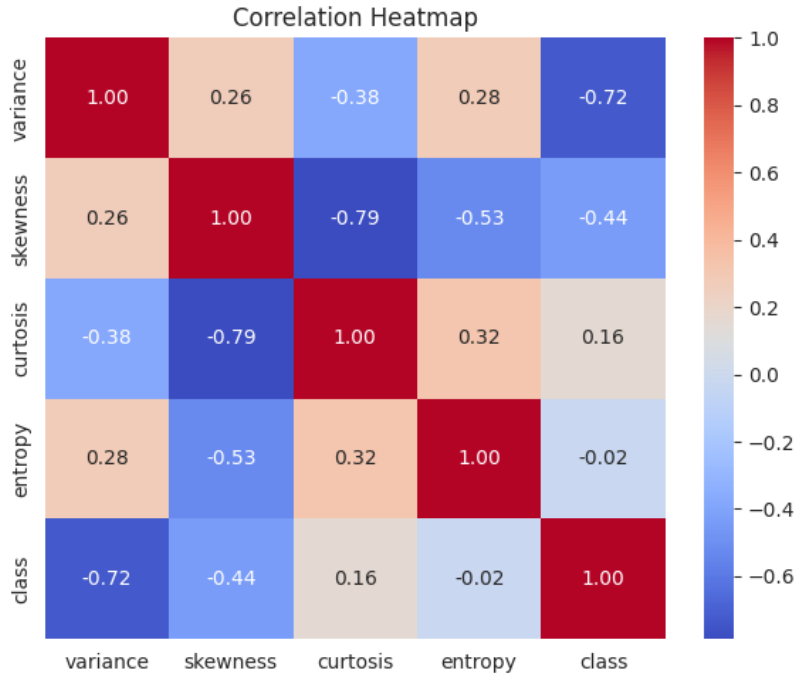


Figure 2: *

Figure 2: Correlation heatmap

Interpretation: Kurtosis and entropy show decent negative correlation whereas variance and skewness are weakly correlated with other features. There are no pairs strongly correlated to each other so as to raise redundancy concerns.

Pairwise Feature Relationships by Banknote Class

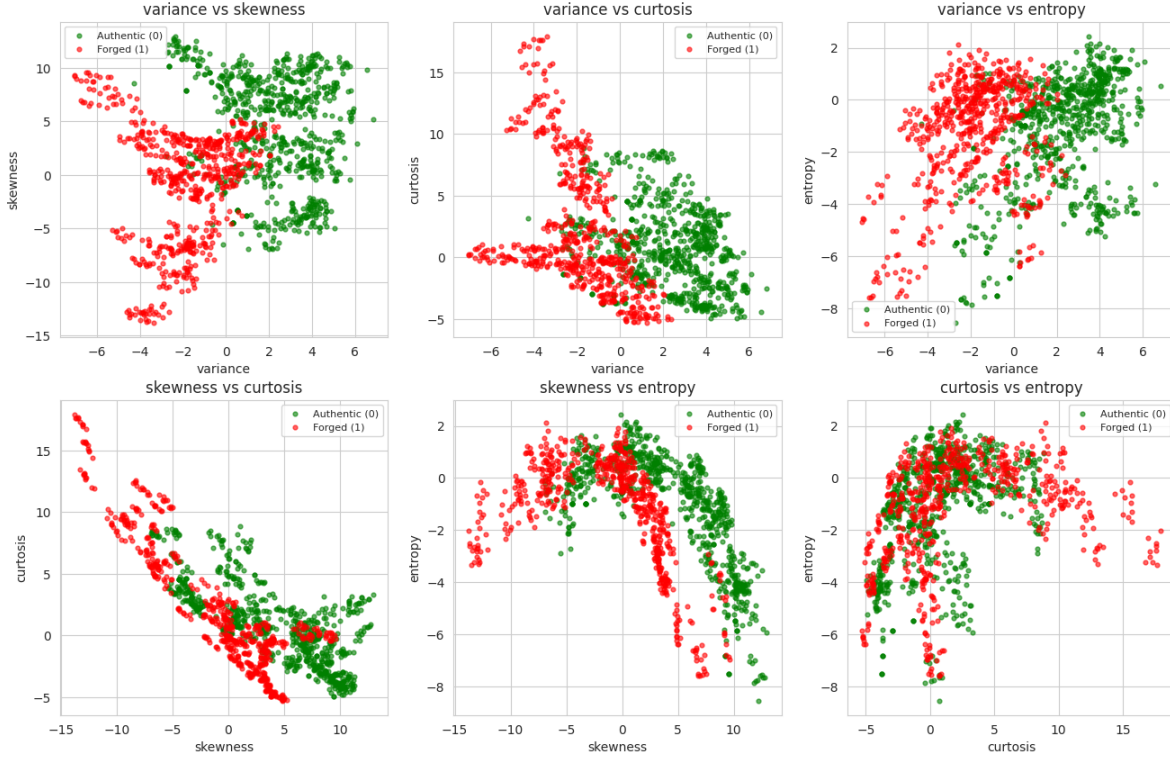


Figure 3: *

Figure 3: Pairwise scatter plots of all feature combinations, coloured by class

Interpretation: Across the six-feature pair combinations, only two classes form visibly distinguishable combinations but the clusters formed are slightly overlapping. Variance-skewness as well as variance-kurtosis show the cleanest separation.

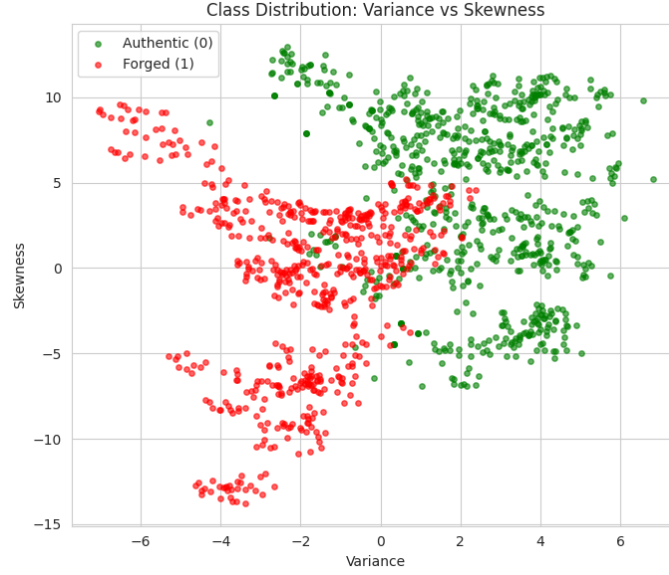


Figure 4: *

Figure 4: Class distribution: variance vs. skewness

Interpretation: This is the most evidently separable pair from all the combinations examined. Authentic data points concentrate at moderate to high variance and skewness whereas Forged data points show lesser variance with a slight presence of linear boundary between them.

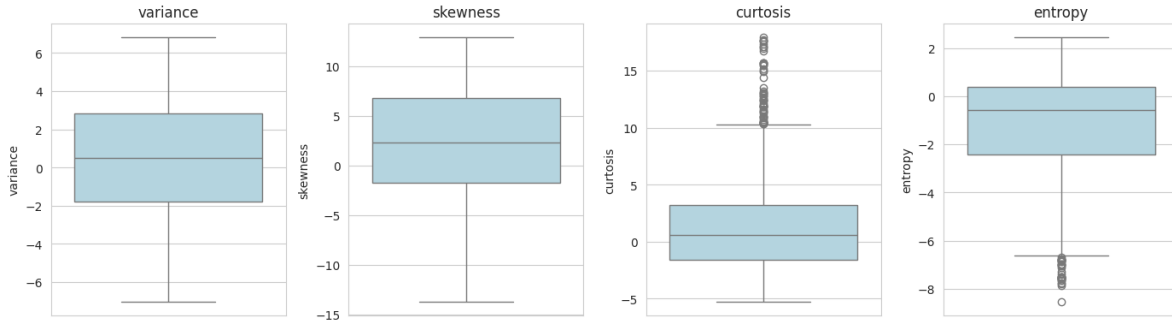


Figure 5: *

Figure 5: Boxplots of the four features

Interpretation: Kurtosis and entropy contain an evident number of outliers beyond the whiskers whereas variance and skewness are compact comparatively which justifies standardizing the features before training.

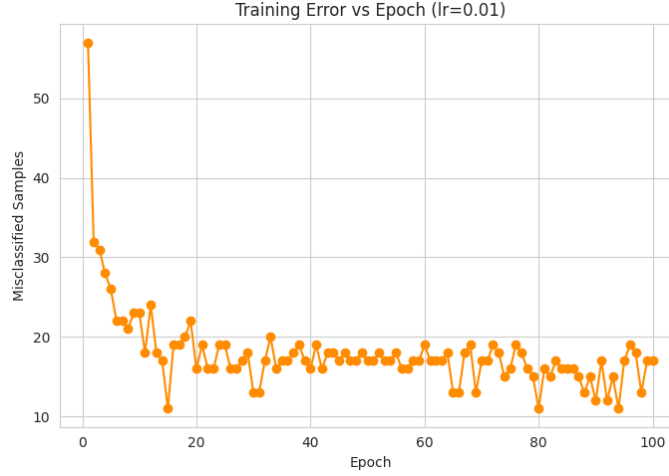


Figure 6: *

Figure 6: Training error (misclassified samples) vs. epoch, $\eta = 0.01$

Interpretation: In the first 10 to 20 epochs, the number of misclassifications decreases drastically, showing minimal oscillations afterwards (approximately 11–19 errors) and supporting the idea that the dataset approaches, but it's not linearly separable.



Figure 7: *

Figure 7: Weight evolution over training epochs

Interpretation: All four weights change fastest during the first 20 epochs and then drift more slowly without fully stabilizing, showing similar values to the residual training error. Variance and skewness end up with the largest magnitude weights.



Figure 8: *

Figure 8: Bias evolution over training epochs

Interpretation: The bias decreases steadily from 0 to approximately -0.12 , shifting the decision boundary to better fit the class distribution in standardized feature space. Like the weights, bias has not fully plateaued by epoch 100.

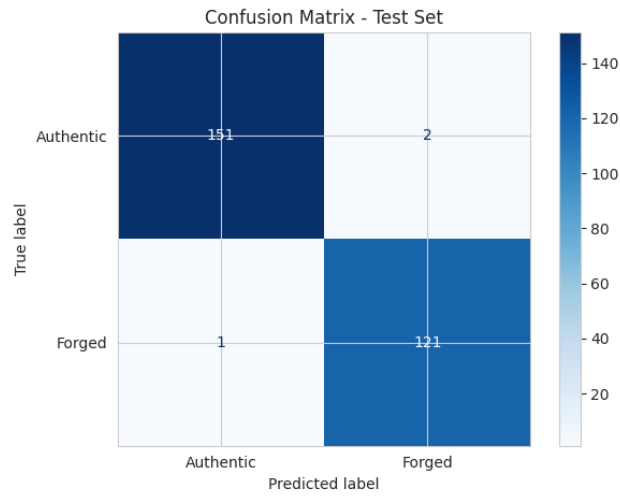


Figure 9: *

Figure 9: Confusion matrix on the test set

Interpretation: The model correctly classifies 151 of 153 authentic notes and 121 of 122 forged notes, with total 3 errors consistent with the high accuracy, precision, and recall reported in Task 6.

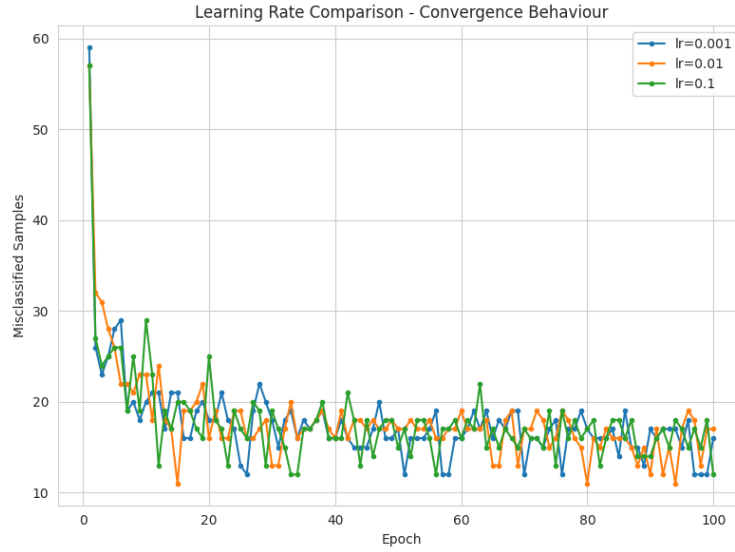


Figure 10: *

Figure 10: Learning rate comparison: convergence behaviour

Interpretation: All three learning rates reduce error quickly at first; $\eta = 0.001$ descends more smoothly and gradually, while $\eta = 0.1$ oscillates more once near the boundary due to larger update steps. $\eta = 0.01$ gives the best balance and the highest test accuracy.

8. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	1097 / 275 (80/20, stratified)
Learning Rate	0.01
Epochs	100 (did not fully converge)
Final Weights	$[-0.2190, -0.2603, -0.2350, -0.0288]$
Final Bias	-0.1200
Accuracy	0.9891
Precision	0.9837
Recall	0.9918
F1-score	0.9878

Epoch-wise Learning

Epoch	Errors	Weight 1 (var.)	Weight 2 (skew.)	Weight 3 (curt.)	Weight 4 (entropy)	Bias
1	57	-0.0623	-0.0697	-0.0650	-0.0025	-0.0300
5	26	-0.1032	-0.1403	-0.1065	0.0074	-0.0400
10	23	-0.1285	-0.1643	-0.1340	-0.0084	-0.0500
20	16	-0.1595	-0.1859	-0.1610	-0.0176	-0.0700
30	13	-0.1744	-0.1934	-0.1793	-0.0225	-0.0800
50	17	-0.1927	-0.2256	-0.2151	-0.0260	-0.0900
80	11	-0.2174	-0.2439	-0.2256	-0.0266	-0.1100
100	17	-0.2190	-0.2603	-0.2350	-0.0288	-0.1200

Learning Rate Comparison

Learning Rate	Epochs Run	Test Accuracy
0.001	100	0.9855
0.01	100	0.9891
0.1	100	0.9855

9. Discussion

The Single Layer Perceptron achieved strong test performance (98.91% accuracy) on the Banknote Authentication dataset because the two classes are close to linearly separable in the standardized 4-dimensional feature space, as confirmed visually by the pairwise scatter plots. However, the training error did not converge to exactly zero within 100 epochs instead of that it oscillated between 11 and 19 misclassifications and after epoch 20 it indicated that a small fraction of samples cannot be separated by any single hyperplane, which means, the data is only linearly separable.

The learning rate comparison confirmed an expected trade-off between convergence speed and stability: $\eta = 0.01$ gave the best balance for this dataset and epoch cost, with $\eta = 0.001$ converging too slowly and $\eta = 0.1$ oscillating more once near the decision boundary.

The from-scratch implementation performed comparably to Scikit-learn’s built-in **Perceptron**, achieving higher precision (0.9837 vs. 0.9683) but slightly lower recall (0.9918 vs. 1.0000) and slightly higher F1-score (0.9878 vs. 0.9839) on this run, which validates the correctness of the manually implemented weight and bias update rules.

10. Source Code

Source code for this experiment can be obtained from this github repository: <https://github.com/vinukohshawenn/layer-perceptron>

11. Conclusion

A Single Layer Perceptron was successfully implemented from scratch and trained on the UCI Banknote Authentication dataset, achieving 98.91% test accuracy, 98.37% precision, 99.18% recall,

and an F1-score of 0.9878. The dataset was found to be approximately linearly separable, allowing the perceptron to perform strongly despite its simple architecture. Learning rate significantly affects convergence behaviour, with $\eta = 0.01$ providing the best balance of speed and stability among the values tested, and the custom implementation was validated against Scikit-learn's Perceptron.

12. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.
2. Compare the implementation with Scikit-learn's Perceptron.
3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.
4. Explain why a Single Layer Perceptron cannot solve the XOR problem.
5. Study the effect of feature normalization on model convergence.

12.1 Answers

1. Step vs. Sigmoid Activation. The Step function has a derivative of zero everywhere except at $z = 0$ (where it is undefined), which makes it incompatible with gradient-based optimization. The Sigmoid function, $\sigma(z) = 1/(1 + e^{-z})$, is smooth and differentiable everywhere and outputs a probability in $(0, 1)$, allowing gradients to flow through multiple layers via backpropagation. This is why modern deep learning models generally tend to use Sigmoid (or ReLU) rather than the Step function.

2. Comparison with Scikit-learn's Perceptron. Scikit-learn's Perceptron was trained with the same learning rate (0.01) and epoch budget (100).

Model	Accuracy	Precision	Recall	F1-score
From-scratch Perceptron	0.9891	0.9837	0.9918	0.9878
Scikit-learn Perceptron	0.9855	0.9683	1.0000	0.9839

The custom implementation achieved slightly better precision and F1-score, while Scikit-learn's version achieved perfect recall (1.0000) but more false positives, which indicates that it is marginally more prone to predict the positive (forged) class on this run. The small differences noted could be due to user implementation such as stopping criteria and so on.

3. Learning Rate Comparison (0.001, 0.01, 0.1). $\eta = 0.001$ produced the smoothest but slowest convergence. $\eta = 0.1$ converged quickly but oscillated more near the decision boundary due to larger update steps. $\eta = 0.01$ gave the best trade-off, achieving the highest test accuracy (98.91%) of the three values tested.

4. Why a Single Layer Perceptron Cannot Solve XOR. XOR is not linearly separable as no single straight line can separate $(0, 0) \rightarrow 0$, $(1, 1) \rightarrow 0$ from $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$. Since a single-layer perceptron with step activation can only represent one linear decision boundary ($\mathbf{w}^T \mathbf{x} + b = 0$), it cannot express the non-linear decision region XOR requires. Solving XOR needs at least one hidden layer (a Multilayer Perceptron).

5. Effect of Feature Normalization on Convergence. Standardizing the features (zero mean, unit variance) puts all four features on a comparable scale before training. Without normalization, features with larger raw magnitudes (e.g., skewness, ranging roughly from -14 to 13)

would dominate the weighted sum and weight updates, dragging and destabilizing convergence. With normalization, the perceptron converges faster and the learned weight magnitudes become directly comparable as indicators of relative feature importance.

Optional Exercise

Implementation of AND, OR, and NOT Gates using a Single Layer Perceptron

As an continuation to the main experiment, a Single Layer Perceptron was trained on the truth tables of the AND, OR, and NOT logic gates to check that a single perceptron can learn any linearly separable Boolean function.

OR Gate

Parameter	Value
Truth Table	$(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 1$
Updates until convergence	5
Final Weights	$[1.0, 1.0]$
Final Bias	-1.0

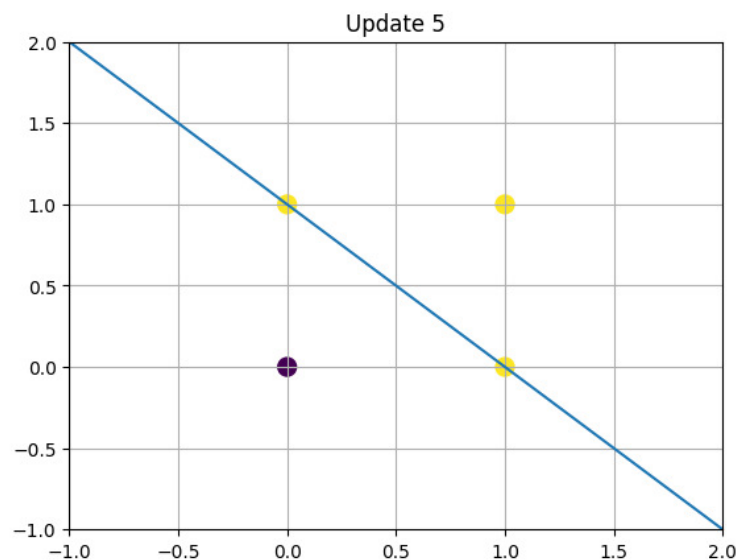


Figure 11: Decision boundary for OR gate

Inference: The perceptron converged, which correctly separates the single negative point $(0, 0)$ from the three positive points using one linear boundary. This shows that the OR logic is linearly separable and is learned exactly by a single-layer perceptron.

AND Gate

Parameter	Value
Truth Table	$(0,0) \rightarrow 0, (0,1) \rightarrow 0, (1,0) \rightarrow 0, (1,1) \rightarrow 1$
Updates until convergence	11
Final Weights	$[2.0, 1.0]$
Final Bias	-3.0

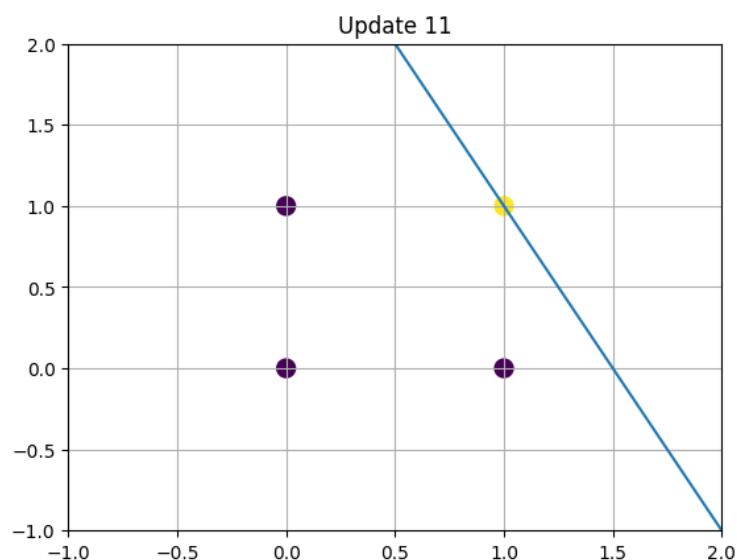


Figure 12: Decision boundary for AND gate

Inference: The perceptron converged which is satisfied only at $(1,1)$ and correctly rejects the other three inputs. AND required more weight updates (11) than OR (5) to converge, since its single positive class occupies a smaller, more tightly bounded region of the input space, making the separating hyperplane harder to locate.

NOT Gate

Parameter	Value
Truth Table	$0 \rightarrow 1, 1 \rightarrow 0$
Updates until convergence	2
Final Weight	$[-1.0]$
Final Bias	0.0

Inference: The perceptron converged to, that is, the output is 1 only when $x = 0$, exactly reproducing the NOT function. Being a single-input, two-point problem, it converged fastest of all three gates (only 2 updates).

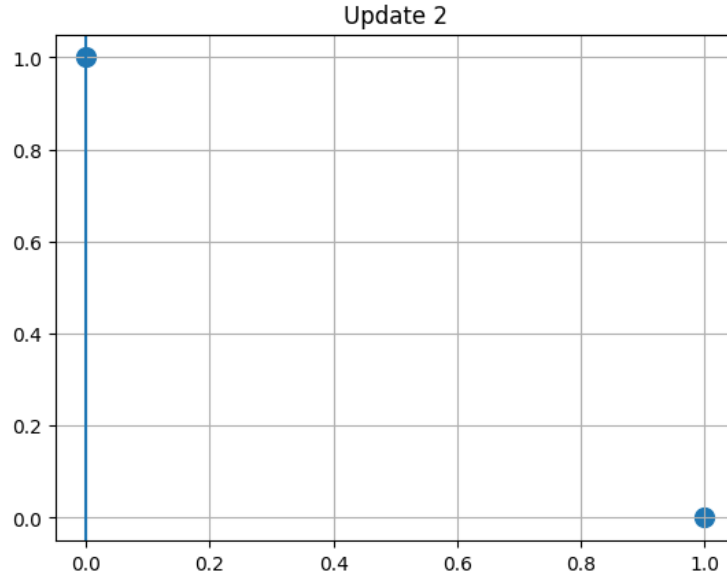


Figure 13: Decision boundary for OR gate

Summary

Gate	Updates to Converge	Final Weights	Final Bias
OR	5	[1.0, 1.0]	-1.0
AND	11	[2.0, 1.0]	-3.0
NOT	2	[-1.0]	0.0

Overall Inference: The three logic gates (OR, AND, and NOT) can be classified as linearly separable Boolean functions by their nature, and therefore, a one-layer perceptron was used to learn the decision boundary for each gate using the perceptron learning algorithm. The number of iterations required for convergence was different for each gate in this experiment, since it depends on a distribution of positive and negative examples relative to the initial weights (a zero vector in our experiment), but not on the Boolean function's complexity..

13. Report Contents

The report should include:

- Objective
- Background Theory
- Activation Functions
- Dataset Description
- Experimental Procedure
- Source Code

- Experimental Results
- Generated Plots with Interpretation
- Performance Tables
- Discussion
- Conclusion
- References

14. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.